종합설계2 · 2026학년도 2학기

시스템 구조 설계 및 개발 환경

System Architecture Design & Development Environments

과목명	종합설계2 (2026학년도 2학기)
문서명	3조 시스템 구조 설계 및 개발 환경
프로젝트명	NISQ 환경에서 QAOA를 이용한 Max-Cut 근사 탐색과 오류 완화 분석
조 / 팀명	3조
조원 (학번·이름·역할)	202302611 정민주(PO) / 202100986 최정호(Dev) / 202303514 최지원(SM)
지도교수	남병규 교수 (서명)
제출일자	2026.10.25
GitHub 저장소	https://github.com/minju43/Team3

Document Revision History

Rev#	Date	Affected Section	Author
1	2026/10/24	최초 작성	최지원
2	2026/10/25	3, 4 작성	정민주

작성·제출 안내

- 본문(0~5 장) 8 쪽 내외 — 표·그림 포함, 부록 제외.
- 파란색 글자는 작성 예시입니다. 제출 전에 모두 삭제하고 우리 팀 내용으로 채우십시오.
- 다이어그램 원본 파일(또는 링크)과 본 문서는 팀 GitHub 저장소에 함께 보관합니다.
- 본 문서는 Sprint 를 진행하며 갱신합니다. 구현과 달라진 부분은 반영하고 Revision History 에 기록합니다.

0. Meta Information

항목	작성 내용
Project	NISQ 환경에서 QAOA를 이용한 Max-Cut 근사 탐색과 오류 완화 분석
Team (팀원 · 역할)	3조 — 정민주 (PO) / 최지원 (SM) / 최정호 (Dev)
Version	v1.0
Framework	Qiskit 1.0+ (QAOA/QPU), M3 & ZNE (Error Mitigation), CVXPY (GW SDP), SciPy (COBYLA), HTML5 Canvas & Chart.js
Scope (담당 US 범위)	US-01 ~ US-08

1. Project Overview

1.1 비전 (Vision)

- 누가(Target) · 어떤 문제(Pain Point) · 어떤 가치(Benefit)를 5 줄 내외로 작성합니다.
- 3 주차에 작성한 HMW(How Might We) 문장의 해답을 문장으로 정리하면 좋습니다.

양자 알고리즘을 학습하는 학생, 연구자와 최적화 문제의 양자 하드웨어 적용 가능성을 검토하는 소프트웨어 엔지니어를 대상으로 NISQ 장비의 노이즈로 인한 QAOA 해 붕괴 및 실질적 유효성 검증의 어려움을 해결한다. 실제 IBM QPU 환경에서 노이즈 왜곡 해와 오류 완화 복원 해를 실시간 비교·시각화하고, 고전 알고리즘 대비 복원율과 근사비를 입증하는 4자 정량 벤치마크 리포트를 제공하여 양자 컴퓨팅의 현실적 유효성을 직관적으로 전달한다.

1.2 범위 (Scope)

구분	내용
In-Scope	· 파티 갈등 문제의 QUBO/Ising 수식 자동 변환 (US-01) · Qiskit 기반 고전-양자 하이브리드 QAOA 최적화 루프 파이프라인 구현 (US-02) · 실제 IBM QPU 백엔드 작업 제출 및 원시 측정 데이터 수집 (US-03) · QPU 측정 결과에 대한 M3 판독 오차 및 ZNE 오류 완화 보정 처리 (US-04) · 비교용 일반 컴퓨터 알고리즘(Brute-force 및 GW 기준선) 개발 (US-05) · 인터랙티브 갈등 관계 웹 캔버스 빌더 구현 (US-06) · 양자 연산 결과 기반 파티 공간 2 분할 시각화 및 회피 갈등 렌더링 (US-07) · 오류 복원 비교 및 4 자 정량 벤치마크 대시보드 표출 (US-08)
Out-of-Scope	· 실시간 트래픽 가로채기 연동 및 동적 라우팅 엔진 · 대규모 노드 대상 양자 연산 지원

1.3 성공 지표 (Success Metrics)

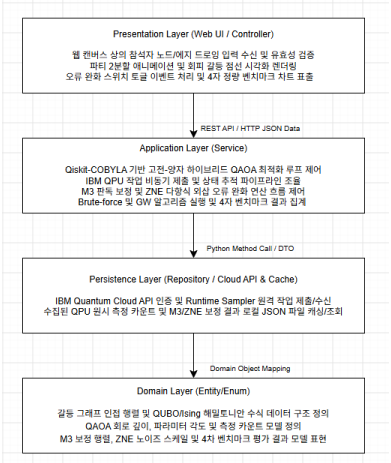
- 정량적으로 측정 가능한 목표를 3~5 개 설정합니다. "안정적인 시스템", "사용하기 편리하게"처럼 판정 기준이 없는 목표는 지표가 아닙니다.

번호	지표	측정 방법	목표치
1	QAOA 근사비 달성률	Brute-force 이론적 최적해 대비 시뮬레이터 기댓값 비율	갈등 그래프(N=8~14) 대상 근사비 ≥ 0.70 달성
2	실제 QPU 오류 완화 성능 복원율	동일 IBM QPU에서 Raw 실행값과 완화 보정값 기댓값 비교	Raw 대비 근사비 또는 정답 비트열 확률 ≥ 20% 향상
3	인터랙티브 웹 시각화 응답 시간	캔버스 파티 2분할 렌더링 시 브라우저 응답 시간 10회 평균	10개 노드 기준 2분할 시각화 및 프리셋 로딩 1.5초 이내
4	4자 벤치마크 파이프라인 정합성 통과율	고전 2종 및 양자 2종 통합 데이터 수집 자동화 테스트 실행	사전 정의 시나리오 대상 테스트 통과율 100%

2. Architecture Design

2.1 논리 아키텍처 (Layered Architecture)

- Controller – Service – Repository – Domain(Entity)의 4 계층 구조를 권장합니다.
- 웹 애플리케이션이 아닌 프로젝트는 계층 이름을 바꿔도 됩니다. 다만 입출력 처리 · 업무 판단 · 데이터 접근을 한 곳에 섞지 않는다는 원칙은 동일하게 지킵니다.



계층별 책임 분담

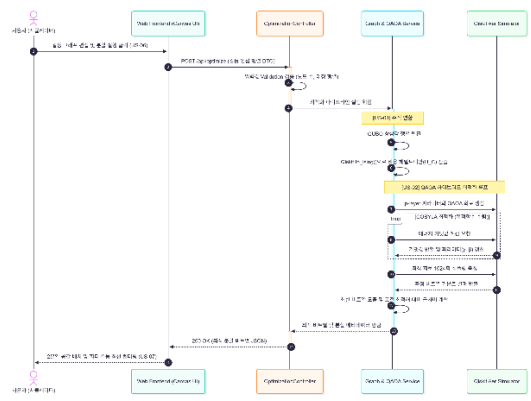
계층	해야 할 일	하지 말아야 할 것	우리 프로젝트에서의 역할
Presentation (Web UI / Controller)	REST 요청·응답 처리, 입력값 유효성 검증, 화면 렌더링	비즈니스 로직 및 알고리즘 계산 직접 수행	노드·에지 입력 검증, 파티 2 분할 점선 렌더링, 오류 완화 토글 및 4 자 벤치마크 차트 표출
Application (Service)	비즈니스 흐름 제어 및 알고리즘 파이프라인 조율	HTTP 서블릿 객체 직접 조작, 외부 API 직접 호출	QAOA 하이브리드 루프 제어, QPU 작업 상태 폴링, M3·ZNE 보정 및 고전(BF·GW) 연산 총괄
Persistence (Repository / Cache)	외부 클라우드 통신, 데이터 저장 및 파일 I/O	비즈니스 규칙 판단 및 가공 로직 포함	IBM Quantum API 인증·작업 제출, QPU 원시 카운트 및 완화 결과 로컬 JSON 캐싱·조회
Domain (Entity / Enum)	핵심 데이터 모델 및 상태 규격 정의	프레임워크 및 외부 통신 모듈에 종속	갈등 인접 행렬, QUBO/Ising 해밀토니안, QAOA 파라미터, 4자 벤치마크 결과 데이터 객체 표현

※ Controller에 if문이 늘어나기 시작하면 설계가 무너지고 있다는 신호입니다. 판단 로직은 Service로 내립니다.

2.2 핵심 업무 흐름 (Core Flow)

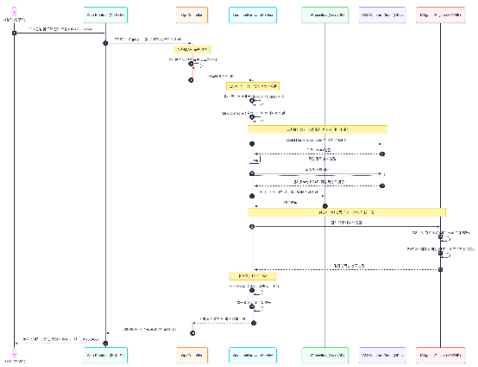
- 시스템의 가장 핵심이 되는 기능 2~3 개에 대해 필수로 작성합니다.
- Sequence Diagram 으로 표현하고, 어려우면 최소한 아래 텍스트 시퀀스 표라도 반드시 작성합니다.
- 코드를 짜기 전에 객체 간의 책임과 상호작용을 미리 검증하기 위한 절차입니다.

Core Flow 1 — 기능명: 파티 갈등 그래프 입력 및 QAOA 하이브리드 최적화



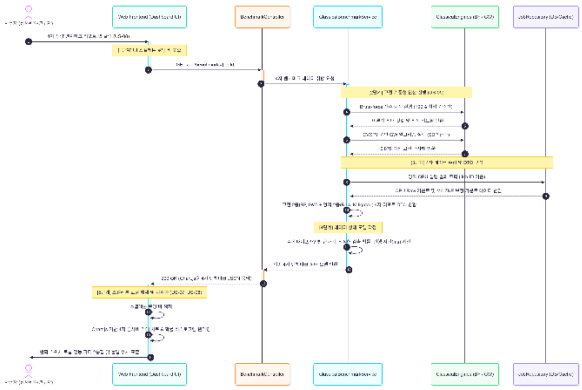
단계	주체(계층)	처리 내용	관련 US
1	Controller	갈등 그래프 편집 요청 수신, 입력값 Validation, DTO 매핑	US-06
2	Service	QUBO 상삼각 행렬 변환 + Qiskit to_ising()을 통한 비용 해밀토니안 산출	US-01
3	Service / Domain	파라미터화된 p-layer QAOA 회로 생성 + COBYLA 최적화 루프 실행	US-02
4	**Service / Domain**	최적 회로 1024회 샘플링 + 최빈 비트열 및 근사비 산출	US-02
5	Controller	최적 파티 배정 비트열 및 근사비 DTO 변환 후 응답 반환	US-02

Core Flow 2 — 기능명: 실제 IBM QPU 백엔드 연동 및 M3/ZNE 오류 완화 보정



단계	주체(계층)	처리 내용	관련 US
1	Controller	실제 QPU 하드웨어 실행 요청 수신, API 토큰 검증	US-03
2	Service	QPU 큐비트 토폴로지 커플링 맵 맞춤 transpile() 수행	US-03
3	Repository	Qiskit Runtime Sampler 기반 비동기 작업 제출 원시 카운트 수신 및 로컬 JSON 캐싱	US-03
4	Service / Domain	M3 판독 오차 보정 + ZNE 2차 다항 외삽 제로 잡음 참값 추정	US-04
5	Controller	사후 정규화(확률합 1.0) 처리된 완화 확률 분포 및 복원 비트열 응답 반환	US-04

Core Flow 3 — 기능명: 고전 기준선 연산 및 4자 정량 벤치마크 대시보드 표출

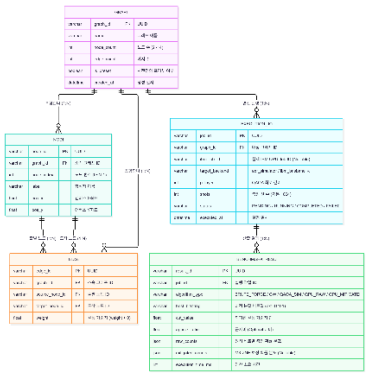


단계	주체(계층)	처리 내용	관련 US
1	Controller	4자 정량 벤치마크 대시보드 조회 요청 수신, UI 스켈레톤 로딩 바 표출	US-08
2	Service	Brute-force 전수 탐색 + CVXPY GW (SDP 완화) 고전 기준선 연산 실행	US-05
3	Service / Domain	고전 2종(BF, GW) 및 양자 2종(Raw, Mitigated) 4자 통합 비교 리포트 DTO 구성	US-05, US-08
4	Service / Domain	4자 근사비, 정답 검출 확률, 연산 수행 시간 데이터 상태 모델 확정	US-05, US-08
5	Controller	스켈레톤 로딩 해제, 보정 스위치 연동 시각화 및 Chart.js 비교 차트 응답 봔환	US-07, US-08

3. 데이터 및 API 설계

3.1 데이터 설계 (ERD)

- 데이터베이스를 사용하는 시스템이라면 ERD 는 필수입니다. 엔티티 / 속성 / 관계(1:N, N:M) / PK · FK 를 모두 표기합니다.
- 객체 구조 설명이 추가로 필요한 경우에만 Class Diagram 을 함께 첨부합니다. (ERD 를 대체할 수 없습니다.)



무결성 규칙 (Integrity Rules)

- 데이터의 비즈니스 규칙을 문장으로 명시합니다. 이 규칙은 곧 Service 계층의 검증 로직과 테스트 케이스가 됩니다.

번호	규칙	적용 위치 (Service / DB 제약)	관련 US
1	노드 수는 2개 이상 14개 이하로 제한한다.	Service 검증 / DB CHECK	US-01, US-06
2	갈등 가중치는 0보다 커야 하며 동일 노드쌍 중복 에지는 불허한다.	DB UNIQUE(graph, src, tgt) / DB CHECK	US-01, US-06
3	분할 결과 비트열 길이는 그래프의 노드 수와 일치해야 한다.	Service 검증	US-02, US-07
4	오류 완화 보정 후 전체 비트열 확률 합은 1.0이어야 한다.	Service 검증	US-04
5	단일 실행 작업 내 알고리즘 유형 결과는 유일해야 한다.	DB UNIQUE(job_id, algo_type)	US-05, US-08

3.2 API 설계 (Endpoint Catalog)

- REST API 형식을 권장합니다. (GraphQL, gRPC 등도 선택 가능하며, 선택한 형식을 먼저 밝힙니다.)
- 기능별 URL · HTTP Method · 설명 · 관련 User Story 를 표로 정리합니다.

API 형식:

Method	Endpoint	설명	관련 US
GET	/api/v1/presets	사전 정의 갈등 관계망 프리셋 목록 조회	US-06, US-09
POST	/api/v1/qaoa/transform	갈등 인접 행렬의 QUBO 및 Ising 수식 자동 변환	US-01
POST	/api/v1/qaoa/solve	무잡음 시뮬레이터 QAOA 최적화 및 최적 비트열 도출	US-02, US-07
GET	/api/v1/qpu/backends	가용 IBM Quantum QPU 백엔드 목록 및 대기열 조회	US-03
POST	/api/v1/qpu/execute	실제 QPU 백엔드 작업 비동기 제출 및 원시 측정 데이터 수집	US-03

Method	Endpoint	설명	관련 US
POST	/api/v1/qpu/mitigate	원시 측정값 대상 M3 판독 오차 및 ZNE 외삽 오류 완화 보정	US-04
POST	/api/v1/classical/benchmark	Brute-force 전수 탐색 및 GW 알고리즘 고전 기준선 산출	US-05
GET	/api/v1/benchmark/dashboard	고전 2종 및 양자 2종 4자 정량 벤치마크 결과 비교 조회	US-08

4. 개발 환경 (Development Environment)

- 팀원 모두가 동일한 환경에서 개발할 수 있도록 버전까지 명시합니다.

항목	우리 팀 설정	주의사항
언어 / 런타임	Python 3.10+	팀원 전원 동일 버전 및 가상환경(venv) 필수 사용
프레임워크	Qiskit SDK 1.0+ & Qiskit Optimization, Qiskit Runtime	버전까지 명시 (Qiskit 1.0+ 호환성 준수)
빌드 도구	pip (pip install -r requirements.txt)	빌드 명령까지 README에 기재
DB	SQLite / JSON·CSV 기반 벤치마크 데이터셋	로컬용 DB 및 JSON 캐시 파일 저장 경로 명시
주요 라이브러리	NetworkX, Qiskit Aer Simulator, Matplotlib & Seaborn, Pytest	추가 시 본 문서 갱신
형상관리	GitHub (소스코드), Notion & Google Docs (산출물 관리)	GitHub 필수 권장, 저장소 주소 기재
브랜치 전략	main / develop / feature-[US번호], PR 1인 이상 리뷰 후 병합	

5. Traceability (추적성)

- 설계가 요구사항을 빠뜨리지 않았는지 확인하는 마지막 단계입니다.
- [양식 5]의 User Story 를 모두 옮겨 적은 뒤, 오른쪽 칸이 비어 있는 행이 없는지 확인합니다.

US 번호	User Story 요약	설계 요소 (Service / Entity / API)	비고
US-01	갈등 그래프 QUBO 수식 변환	QaoaOptimizationService + GraphData(Entity) + POST	

US 번호	User Story 요약	설계 요소 (Service / Entity / API)	비고
		/api/v1/qaoa/transform	
US-02	QAOA 최적화 루프 구축	QaoaOptimizationService + QaoaJobResult(Entity) + POST /api/v1/qaoa/solve	
US-05	GW/Brute-force 산출	QaoaOptimizationService + QaoaJobResult(Entity) + POST /api/v1/qaoa/solve	
US-03	실제 IBM QPU 백엔드 연동 및 측정	QpuExecutionService + QaoaJobResult(Entity) + POST /api/v1/qpu/execute	
US-04	M3 및 ZNE 오류 완화 보정	ErrorMitigationService + MitigatedResult(Entity) + POST /api/v1/qpu/mitigate	
US-06	갈등 그래프 웹 캔버스 빌더	GraphCanvasController + GraphData(Entity) + GraphCanvas (UI)	
US-07	파티 2분할 공간 시각화	PartitionVisualizationController + MitigatedResult(Entity) + PartitionViewer (UI)	
US-08	오류 복원 및 4자 벤치마크 대시보드	BenchmarkService + BenchmarkReport(Entity) + GET /api/v1/benchmark/dashboard	
US-09	대표 갈등 시나리오 프리셋 제공	PresetGraphService + GraphData(Entity) + GET /api/v1/presets	
US-10	QPU 비동기 실행 상태 실시간 알림	QpuStatusService + QaoaJobResult(Entity) + GET /api/v1/qpu/jobs/{job_id}/status	
US-11	4자 벤치마크 결과 CSV/JSON 추출	ReportExportService + BenchmarkReport(Entity) + POST /api/v1/benchmark/export	
US-12	노이즈 시뮬레이션 파라미터 미세 조정 UI	NoiseConfigService + NoiseConfig(Entity) + NoiseConfigPanel (UI)	

6. 제출 전 자가점검 체크리스트

확인	점검 항목
☑	[내용] Vision · Scope · Success Metrics가 [양식 5] Product Backlog와 일치하는가
☑	[내용] Out-of-Scope가 [양식 5]의 Won't 목록과 일치하는가
☑	[내용] Success Metrics 3~5개가 모두 정량적으로 측정 가능한가
☑	[내용] 4계층 구조와 각 계층의 책임 · 금지사항이 우리 프로젝트에 맞게 설명되어 있는가
☑	[내용] 핵심 업무 흐름 2~3개를 시퀀스(다이어그램 또는 표)로 표현했는가
☑	[내용] ERD가 포함되어 있고 PK · FK · 관계가 모두 표기되어 있는가
☑	[내용] 무결성 규칙이 문장으로 명시되고, 적용 위치(Service / DB)가 지정되어 있는가
☑	[내용] Endpoint Catalog가 작성되어 있고 모든 API에 관련 US 번호가 있는가
☑	[내용] 개발 환경이 버전까지 명시되어 있고 형상관리·브랜치 전략이 합의되어 있는가
☑	[내용] [양식 5]의 모든 User Story가 Traceability 표에 매핑되어 있는가
☑	[형식] 파란색 예시 문구를 모두 삭제하고 우리 프로젝트 내용으로 채웠는가
☑	[형식] 표지의 조·조원·역할(PO/SM/Dev)·제출일자·GitHub 저장소를 기재했는가
☑	[형식] Document Revision History를 갱신했는가
☑	[형식] 다이어그램 원본 파일(또는 링크)을 GitHub 저장소에 함께 올렸는가